Домашнее задание по архитектурам №4

Куртеев Александр Михайлович (Б01-407)
kurteev.am@phystech.edu

1. Описание политик и алгоритма

- **Политика замещения LRU (Least Recently Used):** Классический алгоритм вытеснения данных. Когда кэш заполняется, алгоритм удаляет строку, к которой процессор не обращался дольше всего. Исходит из принципа временной локальности: недавно использованные данные понадобятся снова, а давно неиспользуемые можно удалить.
- **Политика замещения SHiP (Signature-based Hit Predictor):** Адаптивный алгоритм, предсказывающий интервал повторного использования кэш линии на основе её сигнатуры (например хэша PC или последовательности инструкций). Обучаясь на истории обращений с помощью таблицы счетчиков (SHCT), SHiP прогнозирует, получит ли строка повторные обращения. Если нет, ей назначается далекий интервал повторного использования для скорейшего вытеснения.
- **Префетчер SPP (Signature Path Prefetcher):** Префетчер, использующий сигнатуру предыдущих доступов внутри физической страницы. SPP находит закономерности в сигнатурах и рекурсивно предсказывает следующие дельты (смещения) обращений. Алгоритм вычисляет уверенность пути и динамически меняет глубину предварительной выборки, чтобы сохранять баланс между покрытием и точностью.

2. Выбранные трассы для анализа

Для экспериментов выбраны следующие трассы:

1. **625.x264_s-18B:** Кодирование видео. Высокая потребность в кэшировании данных и строгая локальность.
2. **445.gobmk-17B:** Частые обращения к сложным структурам данных.
3. **603.bwaves_s-1080B:** Сложные вычисления, требующие больших объемов данных и частых обращений в память. Оно нагружает кэш и может быть полезно для анализа работы L2 кэша в условиях вычислительных интенсивных задач

и конфигурации:

1. **champsim_lru_no_256s_8w**
2. **champsim_lru_no_512s_8w**
3. **champsim_lru_no_512s_16w**
4. **champsim_lru_no_1024s_8w**
5. **champsim_lru_no_1024s_16w**
6. **champsim_ship_no_256s_8w**
7. **champsim_ship_no_512s_8w**
8. **champsim_ship_no_512s_16w**
9. **champsim_ship_no_1024s_8w**
10. **champsim_ship_no_1024s_16w**
11. **champsim_ship_spp_256s_8w**
12. **champsim_ship_spp_512s_8w**
13. **champsim_ship_spp_512s_16w**

14. **champsim_ship_spp_1024s_8w**
15. **champsim_ship_spp_1024s_16w**

3. Методика проведения экспериментов

Моделирование и сбор данных выполнялись при помощи потактового симулятора `ChampSim`. Ниже приведены технические характеристики системы:

- **Процессор:** Intel Core i5-12450H (архитектура Alder Lake-H).
- **Конфигурация ядер:** 8 ядер / 12 потоков (4 высокопроизводительных P-ядра и 4 энергоэффективных E-ядра).
- **Тактовая частота:** Базовая частота P-ядер 2.0 ГГц с возможностью повышения до 4.4 ГГц в режиме Turbo Boost.
- **Параметры кэш-памяти:** L3 Smart Cache — 12 МБ, L2 — 7 МБ.
- **Оперативная память:** 16 гб DDR4.
- **Среда выполнения:** Arch Linux

С целью предотвращения переполнения виртуальной памяти количество одновременно запущенных процессов симуляции было ограничено до 4 единиц с помощью утилиты `GNU Parallel`.

4. Среднее геометрическое

Политика	**128K**	**256K**	**512K (8w)**	**512K (16w)**	**1024K**
LRU	0.6122	0.6343	0.6378	0.6382	0.6398
SHIP	0.1436	0.2612	0.3815	0.2682	0.3852
SHIP+SPP	0.1949	0.2644	0.3406	0.2687	0.3463

Таблица 1: Геометрическое среднее L2 Hit Rate по всем трассам

5. График зависимостей L2_Hit_Rate от размера кэша

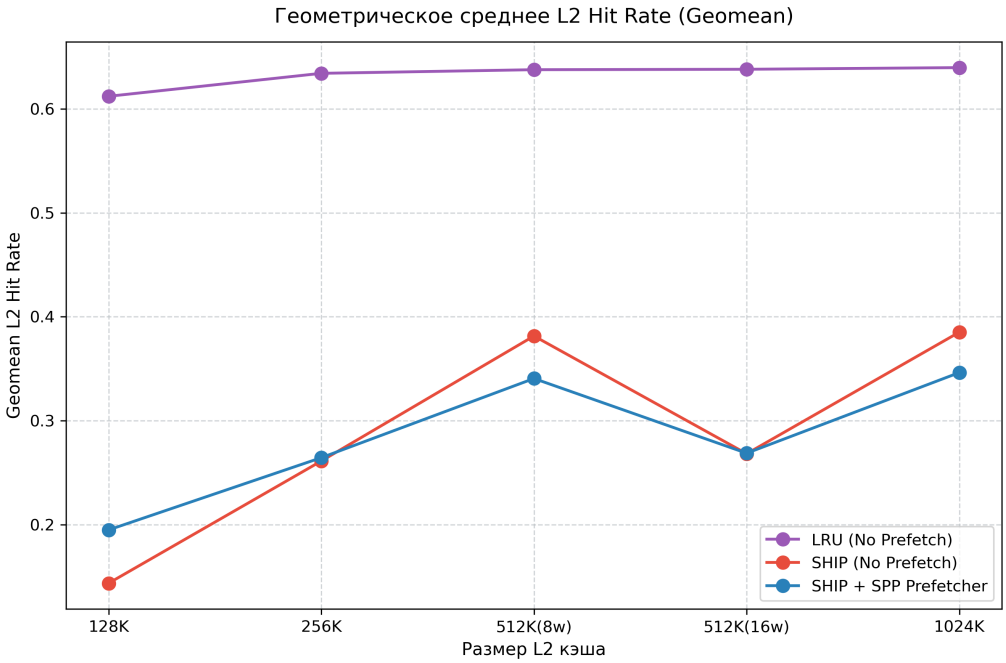

Рис. 1: Зависимость L2 Hit Rate от размера кэша для политик LRU, SHiP и SHiP+SPP.

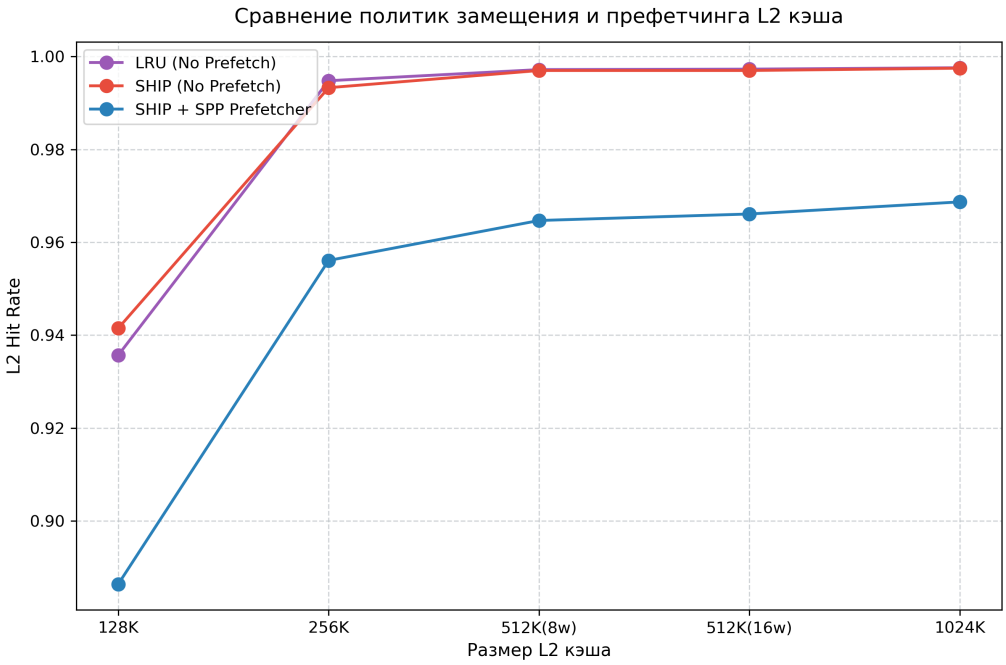

Рис. 2: Зависимость Hit Rate от размера L2 кэша для трассы 445 (gobmk)

6. Выводы по результатам моделирования

На основе геометрического среднего L2 Hit Rate по трем трассам сделаны следующие выводы:

- **Влияние политики SHiP:** На основе полученных данных можно сделать вывод, что смена политики замещения с LRU на SHIP для данного набора трасс оказалась крайне неудачной. Вместо ожидаемого прироста производительности за счет предсказания с сигнатурами повторного использования данных, SHIP показал деградацию Hit Rate. Возможно повторные обращения происходили слишком поздно, поэтому ship помечал их как ненужные.

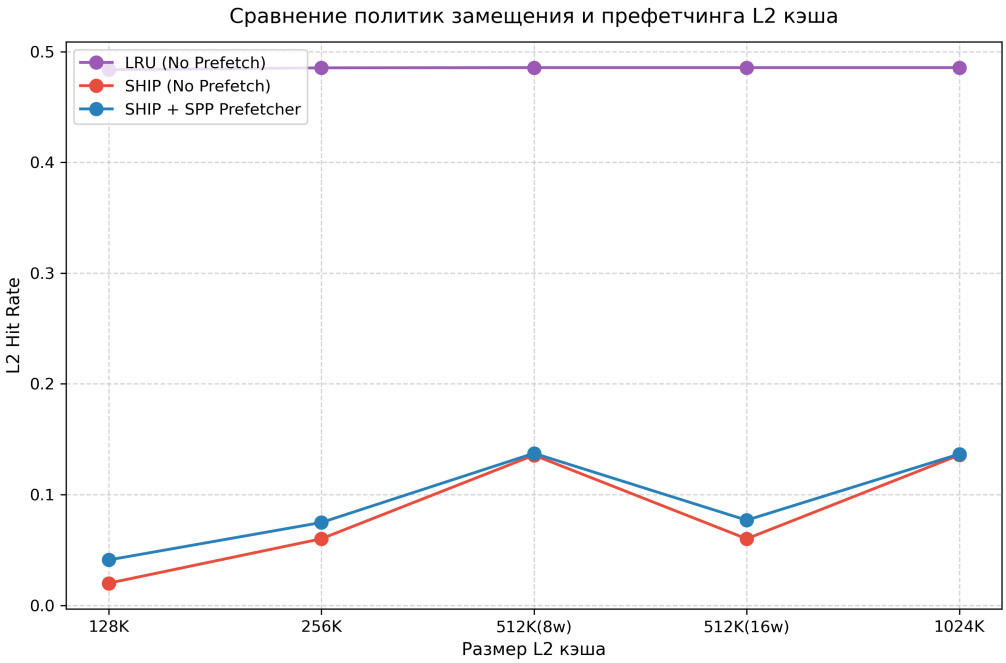

Рис. 3: Зависимость Hit Rate от размера L2 кэша для трассы 603 (bwaves)

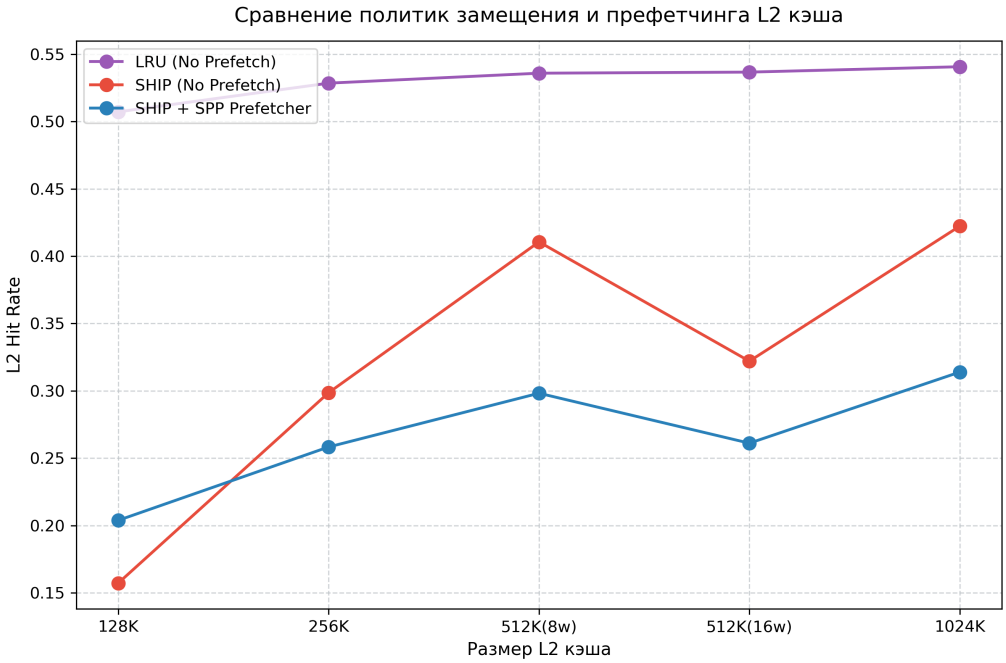

Рис. 4: Зависимость Hit Rate от размера L2 кэша для трассы 625 (x264)

- **Анализ влияния префетчера SPP на производительность**

  Вопреки ожиданиям, на большинстве конфигураций добавление префетчера не только не дало существенного прироста, но и привело к деградации Hit Rate из-за загрязнения кэша. Возможно обращения к данным дыли слишком хаотичными. Так же это могло привести к тому что SPP сбивал с толку SHIP неправильными данными о префетчинге, и SHIP неправильно расставил приоритеты

  – **Отрицательный эффект на трассе 445.gobmk:** На данной нагрузке добавление SPP привело к снижению Hit Rate. В конфигурации 1024K результат упал с 0.9975

(SHIP без префетчера) до 0.9687 (с префетчером). Это свидетельствует о том, что алгоритм SPP генерирует избыточные запросы, которые вытесняют полезные данные из кэша.

- **Локальный прирост на малых объемах:** Положительный эффект наблюдается только при минимальных размерах кэша (128K–256K). Например, на трассе 625.x264 при 128K Hit Rate вырос с 0.15 до 0.20. Однако с ростом объема кэша до 1024K этот выигрыш нивелируется, и голый SHIP начинает работать эффективнее.
- **603.bwaves:** Для потоковой нагрузки bwaves включение префетчера дает крайне малый прирост на малых объемах (с 0.02 до 0.04), который практически исчезает при достижении объема 512K–1024K.
  * На объеме 128K зафиксирован небольшой рост Geomean с 0.14 до 0.19.
  * На объемах 512K и 1024K наблюдается **падение** показателя с 0.38 до 0.34.
- **Анализ через среднее геометрическое:** Агрегированные данные подтверждают, что префетчер SPP оказался скорее вреден:

Итог: Включение префетчера SPP оправдано только в условиях жесткого дефицита памяти (128K). В остальных случаях агрессивные предсказания префетчера засоряют кэш данными, которые не используются повторно, что снижает общую эффективность системы.

- **Определение оптимального размера кэша**

  На основе анализа графиков и средних геометрических показателей можно сделать вывод, что достаточным размером L2 кэша для исследуемого набора трасс является **512K**.

  - Для трассы `445` точка насыщения достигается уже на **256K**, после чего Hit Rate стабилизируется на уровне 99.7%.
  - Для трасс `603` и `625` значимый прирост наблюдается до отметки **512K**. Дальнейшее удвоение объема памяти до 1024K дает прирост эффективности менее 1%.

  Таким образом, дальнейшее увеличение объема кэша сверх 512K не приводит к существенному улучшению.

- **Что выгоднее увеличивать — значение set или way при равном размере кэша**

  При равном объеме кэша 512K сравнение конфигураций 512x16 и 1024x8 показывает следующее:

  - **1024 sets, 8-way:** Является более предпочтительной для большинства исследованных трасс. Увеличение количества наборов снижает вероятность коллизий при отображении адресов, в то время как ассоциативности 8-way достаточно для уменьшения конфликтных промахов.
  - **512 sets, 16-way:** Высокая ассоциативность не дает значимого прироста Hit Rate (разница менее 0.01%), но увеличивает сложность аппаратной реализации и время доступа к данным.

  Результаты моделирования подтверждают, что избыточная ассоциативность (16-way) может негативно сказываться на точности алгоритмов предикта, таких как SHIP, вызывая деградацию Hit Rate относительно более простых конфигураций.

7. Самое интересное задание

Самыми интересными для меня показались первое и четвёртое задания